UT2. PROGRAMACIÓN MULTIHILO

ACTIVIDAD 4

Comunicación entre hilos

Título: Comunicación entre hilos: productor-consumidor con BlockingQueue y con wait()/notify()

Descripción:

Vas a programar el problema clásico **productor-consumidor**. Un hilo produce números y otro los consume, y se los pasan a través de una cola compartida. Primero lo harás con una BlockingQueue, que se encarga ella sola de la sincronización. Después analizarás el mismo problema resuelto a mano con wait() y notify().

Objetivos:

- Compartir información entre hilos mediante una cola bloqueante (**BlockingQueue**).
- Comprobar cómo **put() y take()** bloquean a un hilo cuando la cola está llena o vacía.
- Comparar la sincronización con BlockingQueue y con **wait()/notify()**.

Instrucciones:

Todas las clases del apartado 1 van en el archivo IntercambioHilosDemoCola.java.

1. **Productor-consumidor con BlockingQueue:**
    - **Clase Productor**, que implemente Runnable:
        - Recibe la cola en el constructor: public Productor(BlockingQueue<Integer> cola).
        - En run(), repite 10 veces: genera un número aleatorio entre 1 y 100 con new Random().nextInt(1, 101), muestra Productor: genero X y lo mete en la cola con put().
    - **Clase Consumidor**, que implemente Runnable:
        - Recibe la cola en el constructor: public Consumidor(BlockingQueue<Integer> cola).
        - En run(), repite 10 veces: saca un número de la cola con take(), muestra Consumidor: proceso X y espera 300 ms con Thread.sleep(300).
    - put(), take() y sleep() lanzan InterruptedException: captúrala con try/catch en run().
    - **Clase principal IntercambioHilosDemoCola:**
        - Muestra tu nombre.
        - Crea la cola compartida con capacidad 3: BlockingQueue<Integer> cola = new ArrayBlockingQueue<>(3);

- Crea un hilo productor y un hilo consumidor, pasando la misma cola a los dos, e inícialos con start().
- Espera a que terminen con join() y muestra Fin del programa.

Ten en cuenta: el consumidor es más lento que el productor. Cuando la cola se llena (3 elementos), put() deja al productor esperando hasta que el consumidor saca un número. Esa espera es la sincronización.

2. **Productor-consumidor con wait() y notify():**
   - Añade al mismo proyecto la clase IntercambioHilosDemoWaitNot.java adjunta en la tarea, ejecútala y analiza su código: la clase Compartido hace a mano lo que en el apartado 1 hacía la BlockingQueue.

Preguntas (responde en 2 o 3 líneas cada una):

1. ¿Qué hace put() cuando la cola está llena? ¿Y take() cuando está vacía?
2. En el apartado 2, ¿qué parte del código hace a mano lo que en el apartado 1 hacía la BlockingQueue?
3. Si productor y consumidor compartieran una variable sin ningún mecanismo de sincronización, ¿qué problemas podrían aparecer?

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Solo funciona el productor o el consumidor.
5	Productor y consumidor funcionan con la cola, pero faltan capturas o el apartado 2.
7	Apartados 1 y 2 con sus capturas. Las preguntas están incompletas o tienen errores.
9	Todo correcto: código, capturas y preguntas razonadas.
10	Todo correcto y, además, incluye una mejora que aporta valor al programa.

Entrega:

i. Incluye aquí el código fuente de IntercambioHilosDemoCola:

```
import java.sql.SQLOutput;
import java.util.Random;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

/**
 * @author Juan Gabriel Galarza Claros
 */

class Productor implements Runnable {
    private BlockingQueue<Integer> cola;

    public Productor(BlockingQueue<Integer> cola) {
        this.cola = cola;
    }
```

```
    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                int numero = new Random().nextInt(1, 101);
                System.out.println("Productor genero " + numero);
                cola.put(numero);
            }
        }catch (InterruptedException e) {
            System.err.println("Productor interrumpido");
        }
    }
}

class Consumidor implements Runnable {
    private BlockingQueue<Integer> cola;

    public Consumidor(BlockingQueue<Integer> cola) {
        this.cola = cola;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                int numero = cola.take();
                System.out.println("Consumidor proceso " + numero);

                Thread.sleep(300);
            }
        }catch(InterruptedException e) {
            System.err.println("Productor interrumpido");
        }
    }
}

public class IntercambioHilosDemoCola {
    public static void main() {
        System.out.println("Autor: Juan Gabriel Galarza Claros");

        BlockingQueue<Integer> cola = new ArrayBlockingQueue<>(3);
        Thread productor = new Thread(new Productor(cola));
        Thread consumidor = new Thread(new Consumidor(cola));

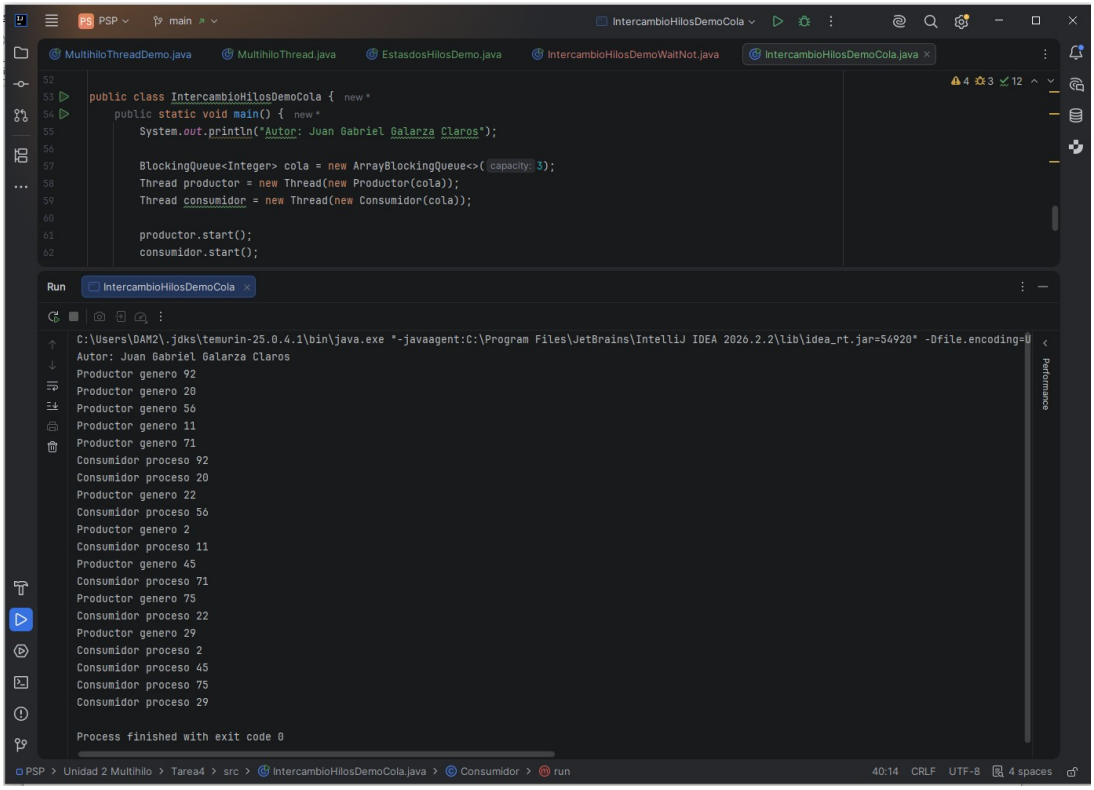
        productor.start();
        consumidor.start();

        try {
            productor.join();
            consumidor.join();

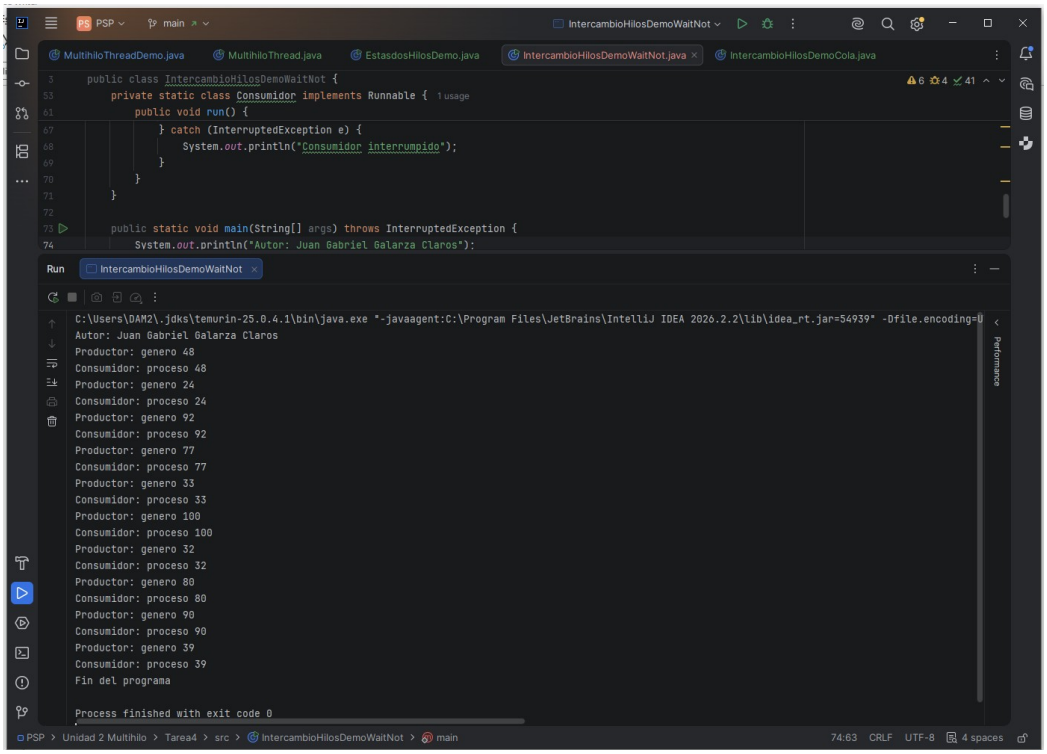
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
```

```
}
```

ii. Incluye aquí la captura de la ejecución de IntercambioHilosDemoCola, donde se vea tu nombre:



iii. Incluye aquí la captura de la ejecución de IntercambioHilosDemoWaitNot, donde se vea tu nombre:



iv. Responde aquí a las preguntas 1, 2 y 3:

1. ¿Qué hace put() cuando la cola está llena? ¿Y take() cuando está vacía?

put() bloquea al hilo productor hasta que se libera espacio en la cola. Y take() bloquea al hilo consumidor hasta que hay al menos un elemento disponible para extraer.

2. En el apartado 2, ¿qué parte del código hace a mano lo que en el apartado 1 hacía la BlockingQueue?

La clase interna Compartido junto con sus métodos sincronizados, que utilizan wait() y notify() para gestionar manualmente el estado de disponibilidad y el bloqueo de los hilos.

3. Si productor y consumidor compartieran una variable sin ningún mecanismo de sincronización, ¿qué problemas podrían aparecer?

Se producirían condiciones de carrera, problemas de visibilidad de memoria entre hilos y pérdida, sobrescritura o lectura duplicada de datos al no haber exclusión mutua.